

Documentazione del codice per la registrazione del phantom cervicale con Deep Closest Point adattato

Progetto di Robotica Medica

22 settembre 2026

Indice

1	Introduzione	2
2	Panoramica dei file	2
3	Modifiche rispetto al DCP originale	2
4	File util.py	3
4.1	Funzioni	3
5	File model.py	3
5.1	Funzioni di supporto	3
5.2	Classi	4
6	File correspondence_loss.py	7
7	File phantom_data.py	7
7.1	Eccezioni	7
7.2	Funzioni	7
7.3	Classe PhantomDataset	8
8	File train.py	8
8.1	Funzioni	8
9	File evaluate.py	9
9.1	Classe IOStream	9
9.2	Funzioni	9
10	File test_correspondence.py	11
11	File unify.py	11

1 Introduzione

Questo documento descrive in dettaglio l'implementazione del progetto di registrazione del phantom cervicale basata su Deep Closest Point (DCP) adattato. Il codice originale di DCP [1] è stato modificato per operare su dati sintetici generati da una mesh STL del phantom, con l'obiettivo di stimare la trasformazione rigida Polaris $\rightarrow$ phantom a partire da acquisizioni parziali (sweep o sparse) senza punti fiduciali prescritti.

La struttura del documento segue l'organizzazione dei file sorgente. Per ogni file vengono elencate le funzioni e le classi, ciascuna corredata da pre-condizioni, post-condizioni e una spiegazione del ruolo. Particolare attenzione è dedicata alla sezione 3, che riassume i cambiamenti rispetto al repository DCP originale.

2 Panoramica dei file

File	Contenuto principale
<code>evaluate.py</code>	Valutazione TRE, baseline SVD a 4 punti, metriche di rotazione e traslazione.
<code>train.py</code>	Loop di training e validazione, gestione checkpoint, loss di corrispondenza.
<code>test_correspondence.py</code>	Test unitari per la loss di corrispondenza, il modello e il dataset.
<code>phantom_data.py</code>	Generazione del dataset sintetico da mesh STL (sweep/sparse).
<code>model.py</code>	Architettura DCP adattata (DGCNN, Transformer, SVD-Head, MLPHead).
<code>correspondence_loss.py</code>	Loss di corrispondenza geometrica basata su KL divergenza.
<code>util.py</code>	Funzioni di utilità (quaternioni, trasformazione di point cloud).
<code>unify.py</code>	Script per unire mesh STL multiple in un unico file.

3 Modifiche rispetto al DCP originale

Il codice originale di DCP [1] è stato adattato per il caso d'uso del phantom cervicale. Le modifiche principali sono:

1. **Dataset sintetico:** sostituzione di ModelNet40 con `PhantomDataset`, che campiona point cloud da una mesh STL e genera trasformazioni rigide note.
2. **GroupNorm in DGCNN:** le BatchNorm sono state sostituite con GroupNorm per garantire stabilità con batch di piccole dimensioni e per evitare dipendenze dalle statistiche del batch durante la valutazione.
3. **Loss di corrispondenza geometrica:** aggiunta della `geometric_correspondence_loss` (KL tra distribuzione geometrica e distribuzione appresa) come supervisione ausiliaria opzionale.
4. **Modalità return_correspondence:** DCP e SVDHead possono restituire i logits delle corrispondenze per il calcolo della loss ausiliaria.
5. **Scala di rete:** introduzione di `network_scale_mm` per normalizzare le coordinate in millimetri in unità della rete, mantenendo la coerenza tra training e valutazione.

6. **Gestione del digital twin:** il riferimento globale (`reference_phantom`) viene generato una sola volta e salvato nel checkpoint per garantire che training, validation e test utilizzino la stessa discretizzazione.
7. **Valutazione TRE:** `evaluate.py` implementa il calcolo del Target Registration Error su target anatomici e la baseline SVD a 4 punti con rumore realistico.
8. **Test unitari:** aggiunta di `test_correspondence.py` per verificare il corretto funzionamento della loss, del modello e del dataset.

4 File `util.py`

4.1 Funzioni

Funzione: `quat2mat`

Pre-condizioni: `quat`: tensore di forma $[B, 4]$ con componenti (x, y, z, w) .

Post-condizioni: Matrice di rotazione $[B, 3, 3]$.

Spiegazione:

Converte un batch di quaternioni in matrici di rotazione. Implementazione vettorizzata basata sulla formula esplicita del quaternione.

Funzione: `transform_point_cloud`

Pre-condizioni: `point_cloud`: $[B, 3, N]$; `rotation`: $[B, 3, 3]$ o $[B, 4]$; `translation`: $[B, 3]$.

Post-condizioni: Point cloud trasformato: $[B, 3, N]$.

Spiegazione:

Applica la trasformazione rigida (R, t) a un point cloud. Se `rotation` ha 4 componenti, viene convertita da quaternione tramite `quat2mat`.

Funzione: `npmat2euler`

Pre-condizioni: `mats`: array $[B, 3, 3]$; `seq`: stringa che specifica l'ordine degli assi per gli angoli di Eulero.

Post-condizioni: Array $[B, 3]$ di angoli di Eulero in gradi.

Spiegazione:

Converte un batch di matrici di rotazione in angoli di Eulero usando `scipy.spatial.transform.Rotation`.

5 File `model.py`

5.1 Funzioni di supporto

Funzione: `clones`

Pre-condizioni: `module`: modulo PyTorch; `N`: numero di copie.

Post-condizioni: `ModuleList` contenente `N` copie indipendenti di `module`.

Spiegazione:

Crea `N` copie profonde di un modulo, utile per impilare layer identici ma con pesi indipendenti.

Funzione: `attention`

Pre-condizioni: `query`, `key`, `value`: tensori $[B, h, N, d_k]$; `mask`: tensore opzionale; `dropout`: tasso di dropout.

Post-condizioni: Output dell'attenzione e matrice dei pesi.

Spiegazione:

Implementa l'attenzione a prodotto scalare scalato (*scaled dot-product attention*) come definita in [4].

Funzione: nearest_neighbor

Pre-condizioni: src, dst: tensori [C, N] e [C, M].

Post-condizioni: Distanze e indici del punto più vicino in dst per ogni punto di src.

Spiegazione:

Calcola il nearest neighbor in modo esplicito espandendo le distanze euclidee al quadrato. Usato solo in contesti diagnostici.

Funzione: knn

Pre-condizioni: x: tensore [B, C, N]; k: numero di vicini.

Post-condizioni: Indici dei k vicini più prossimi: [B, N, k_eff].

Spiegazione:

Costruisce il grafo dei k-nearest neighbors. Il punto stesso è incluso. Se $k > N$, viene limitato a N.

Funzione: get_graph_feature

Pre-condizioni: x: tensore [B, C, N]; k: numero di vicini.

Post-condizioni: Tensore [B, 2*C, N, k_eff] contenente le feature di bordo.

Spiegazione:

Costruisce le feature di bordo concatenando le coordinate dei vicini e del centro, come nell'implementazione originale di DGCNN.

Funzione: make_group_norm

Pre-condizioni: num_channels: numero di canali.

Post-condizioni: Modulo GroupNorm configurato con il maggior numero di gruppi che divide esattamente i canali.

Spiegazione:

Sostituisce le BatchNorm originali con GroupNorm per migliorare la stabilità su batch piccoli.

5.2 Classi

Classe: EncoderDecoder

Descrizione: Architettura encoder-decoder basata su Transformer, usata dal modulo Transformer per raffinare le feature tramite self-attention e cross-attention.

Metodi:

- **__init__** — *Pre:* encoder, decoder, src_embed, tgt_embed, generator: moduli PyTorch.; *Post:* Inizializza i componenti.
- **forward** — *Pre:* src, tgt, src_mask, tgt_mask.; *Post:* Output del decoder dopo encoding e decoding.
- **encode** — *Pre:* src, src_mask.; *Post:* Memoria codificata.
- **decode** — *Pre:* memory, src_mask, tgt, tgt_mask.; *Post:* Output del generatore.

Classe: Generator

Descrizione: Rete fully-connected che produce una stima diretta di rotazione (quaternione) e traslazione a partire da un embedding globale. Usata solo con head=mlp.

Metodi:

- **__init__** — *Pre:* emb_dims: dimensione dell'embedding.; *Post:* Inizializza i layer lineari e le proiezioni.

- **forward** — *Pre:* x : embedding $[B, D, N]$; *Post:* Coppia (quaternion normalizzato, traslazione).

Classe: Encoder

Descrizione: Stack di N layer encoder con LayerNorm finale.

Metodi:

- **__init__** — *Pre:* layer, N ; *Post:* Clona il layer N volte.
- **forward** — *Pre:* x , mask.; *Post:* Feature codificate.

Classe: Decoder

Descrizione: Stack di N layer decoder con LayerNorm finale.

Metodi:

- **__init__** — *Pre:* layer, N ; *Post:* Clona il layer N volte.
- **forward** — *Pre:* x , memory, src_mask, tgt_mask.; *Post:* Feature decodificate.

Classe: LayerNorm

Descrizione: Layer normalization personalizzata con parametri learnable γ e β .

Metodi:

- **__init__** — *Pre:* features, eps.; *Post:* Inizializza parametri e epsilon.
- **forward** — *Pre:* x .; *Post:* Tensore normalizzato.

Classe: SublayerConnection

Descrizione: Connessione residua con LayerNorm, usata nei layer encoder e decoder.

Metodi:

- **__init__** — *Pre:* size, dropout.; *Post:* Inizializza LayerNorm.
- **forward** — *Pre:* x , sublayer.; *Post:* $x + \text{sublayer}(\text{LayerNorm}(x))$.

Classe: EncoderLayer

Descrizione: Layer encoder composto da self-attention e feed-forward.

Metodi:

- **__init__** — *Pre:* size, self_attn, feed_forward, dropout.; *Post:* Inizializza i sublayer.
- **forward** — *Pre:* x , mask.; *Post:* Output del layer.

Classe: DecoderLayer

Descrizione: Layer decoder composto da self-attention, cross-attention e feed-forward.

Metodi:

- **__init__** — *Pre:* size, self_attn, src_attn, feed_forward, dropout.; *Post:* Inizializza i sublayer.
- **forward** — *Pre:* x , memory, src_mask, tgt_mask.; *Post:* Output del layer.

Classe: MultiHeadedAttention

Descrizione: Attenzione multi-head con proiezioni lineari per Q, K, V e output.

Metodi:

- **__init__** — *Pre:* h , d_{model} , dropout.; *Post:* Inizializza le proiezioni.
- **forward** — *Pre:* query, key, value, mask.; *Post:* Output dell'attenzione multi-head.

Classe: PositionwiseFeedForward

Descrizione: Rete feed-forward a due strati applicata indipendentemente a ciascun punto.

Metodi:

- **__init__** — *Pre:* d_{model} , d_{ff} , dropout.; *Post:* Inizializza i layer lineari.

- **forward** — *Pre:* x.; *Post:* Output del feed-forward.

Classe: PointNet

Descrizione: Encoder PointNet con 5 convoluzioni 1D e BatchNorm. Usato quando `emb_nn=pointnet`.

Metodi:

- **__init__** — *Pre:* emb_dims.; *Post:* Inizializza convoluzioni e BatchNorm.
- **forward** — *Pre:* x: [B, 3, N].; *Post:* Embedding [B, emb_dims, N].

Classe: DGCNN

Descrizione: Encoder DGCNN con 5 convoluzioni 2D e GroupNorm. Usato quando `emb_nn=dgcnn`.

Metodi:

- **__init__** — *Pre:* emb_dims, k.; *Post:* Inizializza convoluzioni e GroupNorm.
- **forward** — *Pre:* x: [B, 3, N].; *Post:* Embedding [B, emb_dims, N].

Classe: MLPHead

Descrizione: Testa di regressione diretta che stima rotazione (quaternione) e traslazione da embedding concatenati.

Metodi:

- **__init__** — *Pre:* args.; *Post:* Inizializza la MLP e le proiezioni.
- **forward** — *Pre:* src_embedding, tgt_embedding.; *Post:* Matrice di rotazione e traslazione.

Classe: Identity

Descrizione: Modulo no-op usato come pointer in DCP-v1.

Metodi:

- **forward** — *Pre:* *input.; *Post:* Restituisce gli input invariati.

Classe: Transformer

Descrizione: Modulo pointer basato su Transformer che permette a ciascun point cloud di arricchire i propri embedding con informazioni dell'altro.

Metodi:

- **__init__** — *Pre:* args.; *Post:* Inizializza encoder, decoder e attention.
- **forward** — *Pre:* src, tgt.; *Post:* Coppia di embedding contestuali.

Classe: SVDHead

Descrizione: Testa che risolve il problema di Procrustes tramite SVD differenziabile per ottenere la trasformazione rigida.

Metodi:

- **__init__** — *Pre:* args.; *Post:* Registra il buffer **reflect** per correggere riflessioni.
- **forward** — *Pre:* src_embedding, tgt_embedding, src, tgt, return_correspondence.; *Post:* Rotazione, traslazione e (opzionalmente) logits delle corrispondenze.

Classe: DCP

Descrizione: Modello completo che combina encoder, pointer e testa per stimare la trasformazione rigida tra due point cloud.

Metodi:

- **__init__** — *Pre:* args.; *Post:* Inizializza encoder, pointer e testa in base agli argomenti.
- **forward** — *Pre:* src, tgt, return_correspondence.; *Post:* Rotazione e traslazione (e logits se richiesto).

6 File `correspondence_loss.py`

Funzione: `geometric_correspondence_loss`

Pre-condizioni: `logits`: $[B, N_s, N_t]$ logits della rete; `src`, `target`: $[B, 3, N_s]$ e $[B, 3, N_t]$; `rotation_gt`, `translation_gt`: trasformazione ground truth; `network_scale_mm`: scala in mm; `sigma_mm`: deviazione standard geometrica in mm.

Post-condizioni: Scalare: KL divergenza media tra distribuzione geometrica e distribuzione appresa.

Spiegazione:

Calcola la loss di corrispondenza geometrica come $KL(q_{GT} || p_{rete})$. Le etichette q_{GT} sono costruite trasformando `src` con la GT e calcolando le distanze da `target`, quindi applicando una softmax con temperatura σ . I logits della rete vengono normalizzati con log-softmax. La loss incoraggia la rete a riprodurre la distribuzione geometrica delle corrispondenze.

7 File `phantom_data.py`

7.1 Eccezioni

Classe: `MeshLoadError`

Descrizione: Eccezione sollevata quando la mesh non può essere caricata o non è valida. Non definisce metodi propri: è una sottoclasse vuota di `Exception`.

Metodi:

- `__init__` — *Pre*: messaggio di errore.; *Post*: Istanza di eccezione con messaggio associato.

7.2 Funzioni

Funzione: `load_mesh`

Pre-condizioni: `stl_path`: percorso al file STL.

Post-condizioni: Oggetto `trimesh.Trimesh` validato.

Spiegazione:

Carica la mesh con `trimesh`, verifica che non sia vuota e che le aree delle facce siano finite e positive. Rimuove facce degeneri se presenti.

Funzione: `build_face_neighbors`

Pre-condizioni: `mesh`: oggetto `Trimesh`.

Post-condizioni: Matrice sparsa CSR delle adiacenze tra facce.

Spiegazione:

Costruisce il grafo delle facce che condividono uno spigolo, rendendolo simmetrico.

Funzione: `_sample_triangles`

Pre-condizioni: `mesh`; `face_ids`: array di indici di facce; `rng`: generatore casuale.

Post-condizioni: Array $[\text{len}(\text{face_ids}), 3]$ di punti campionati uniformemente sulle facce.

Spiegazione:

Campiona un punto uniforme in ciascuna faccia selezionata usando coordinate baricentriche.

Funzione: `farthest_point_sample`

Pre-condizioni: `points`: $[N, 3]$; `n_points`: numero di punti da selezionare; `rng`.

Post-condizioni: Array $[n_points, 3]$ di punti selezionati.

Spiegazione:

Seleziona `n_points` punti distribuiti uniformemente tramite Farthest Point Sampling. Utile per costruire un digital twin ben distribuito.

Funzione: `sample_mesh_surface`

Pre-condizioni: `mesh`; `n_points`; `strategy`: 'random' o 'patch'; `rng`; `face_neighbors`; `face_probabilities`; `patch_radius_mm`.

Post-condizioni: Array [`n_points`, 3] di punti sulla superficie.

Spiegazione:

Campiona punti sulla superficie della mesh. Se `strategy=patch`, costruisce una patch connessa entro un raggio specificato; altrimenti campiona casualmente in base all'area delle facce.

Funzione: `_sample_patch`

Pre-condizioni: `mesh`; `n_points`; `rng`; `face_neighbors`; `face_probabilities`; `patch_radius_mm`.

Post-condizioni: Array [`n_points`, 3] di punti nella patch.

Spiegazione:

Costruisce una patch connessa a partire da una faccia seme, includendo le facce raggiungibili entro `patch_radius_mm`. Campiona quindi `n_points` punti all'interno della patch.

Funzione: `generate_transform`

Pre-condizioni: `rng`; `rot_max`; `trans_max`; `euler_seq`.

Post-condizioni: Tuple (`R_ab`, `t_ab`, `R_ba`, `t_ba`).

Spiegazione:

Genera una trasformazione rigida casuale con rotazioni limitate per asse e traslazioni limitate per asse. Restituisce anche l'inversa.

7.3 Classe PhantomDataset

Classe: `PhantomDataset`

Descrizione: Dataset PyTorch che genera coppie di point cloud (`src`, `tgt`, `R`, `t`) da una mesh STL. Supporta due modalità: **sweep** (patch connessa) e **sparse** (punti sparsi).

Metodi:

- `__init__` — *Pre:* `stl_path`, `mode`, `num_samples`, `n_points`, `target_n_points`, `rot_max`, `trans_max`, `noise_sigma`, `factor`, `seed`, `network_scale_mm`, `patch_radius_mm`; *Post:* Carica la mesh, valida i parametri, genera il digital twin di riferimento e prepara le probabilità delle facce. Se `mode=sweep`, costruisce le adiacenze.

- `__len__` — *Pre:* ; *Post:* Numero di campioni.

- `__getitem__` — *Pre:* `item`: indice.; *Post:* Tuple (`src_tensor`, `tgt_tensor`, `R_gt_tensor`, `t_gt_tensor`). Genera una coppia di point cloud con trasformazione nota, applica rumore e normalizza le coordinate.

8 File train.py

8.1 Funzioni

Funzione: `_init__`

Pre-condizioni: `args`: namespace degli argomenti.

Post-condizioni: Crea la directory dell'esperimento e copia i file sorgente come backup.

Spiegazione:

Verifica che non esistano già checkpoint (a meno che non si usi `-resume`), crea la cartella `checkpoints/<exp_name>/models` e salva copie dei file sorgente.

Funzione: `predict_with_aux_loss`

Pre-condizioni: args; net; src, target, rotation_gt, translation_gt.

Post-condizioni: Tuple (predizioni, corr_loss).

Spiegazione:

Se `corr_weight > 0`, esegue il forward con `return_correspondence=True` e calcola la loss di corrispondenza geometrica. Altrimenti restituisce le predizioni standard e una loss nulla.

Funzione: `train_one_epoch`

Pre-condizioni: args; net; train_loader; optimizer; textio; epoch.

Post-condizioni: Dizionario di metriche di training.

Spiegazione:

Esegue un'epoca di training. Calcola la loss di pose (MSE su rotazione e traslazione), opzionalmente la cycle consistency loss e la loss di corrispondenza. Esegue backpropagation con clipping dei gradienti. Restituisce metriche medie pesate per batch.

Funzione: `validate_one_epoch`

Pre-condizioni: args; net; test_loader; textio; epoch.

Post-condizioni: Dizionario di metriche di validazione.

Spiegazione:

Esegue la validazione in modalità `eval()`. Calcola le stesse metriche del training ma senza aggiornare i pesi.

Funzione: `main`

Pre-condizioni:

Post-condizioni:

Spiegazione:

Punto di ingresso del training. Gestisce il parsing degli argomenti, la validazione dei parametri, la creazione dei dataset, il modello, l'ottimizzatore e lo scheduler. Esegue il loop di training per il numero di epoche specificato, salva i checkpoint (ultimo e migliore) e registra le metriche su TensorBoard.

9 File `evaluate.py`

9.1 Classe `IOStream`

Classe: `IOStream`

Descrizione: Classe per la scrittura di log su file e console.

Metodi:

- `__init__` — *Pre:* path: percorso del file di log (o None).; *Post:* Apre il file in modalità append se il percorso è fornito.
- `cprint` — *Pre:* text: stringa.; *Post:* Stampa il testo e lo scrive nel file se aperto.
- `close` — *Pre:* ; *Post:* Chiude il file.

9.2 Funzioni

Funzione: `apply_rigid_transform`

Pre-condizioni: points: [N, 3]; R: [3, 3]; t: [3].

Post-condizioni: Punti trasformati [N, 3].

Spiegazione:

Applica la trasformazione rigida $p' = Rp + t$ a ciascun punto.

Funzione: `fit_rigid_svd`

Pre-condizioni: `src_pts`, `tgt_pts`: $[N, 3]$ con $N \geq 3$.

Post-condizioni: Coppia (R, t) che mappa `src_pts` su `tgt_pts`.

Spiegazione:

Stima la trasformazione rigida tramite SVD (soluzione di Procrustes). Verifica che i punti non siano degeneri ($\text{rango} \geq 2$) e che il determinante di R sia $+1$.

Funzione: `rotation_error_deg`

Pre-condizioni: `R_pred`, `R_gt`: $[3, 3]$.

Post-condizioni: Errore angolare in gradi.

Spiegazione:

Calcola l'angolo della rotazione relativa $R_{pred}R_{gt}^T$.

Funzione: `translation_error_mm`

Pre-condizioni: `t_pred_mm`, `t_gt_mm`: $[3]$.

Post-condizioni: Distanza euclidea in mm.

Spiegazione:

Calcola la norma della differenza tra le traslazioni.

Funzione: `compute_tre`

Pre-condizioni: `R_pred`, `t_pred_mm`, `R_gt`, `t_gt_mm`, `target_pts_phantom_mm`: $[N, 3]$.

Post-condizioni: Array $[N]$ di errori TRE in mm.

Spiegazione:

Calcola il Target Registration Error per ciascun target. I target sono definiti nel frame phantom; la trasformazione GT viene usata per portarli nel frame Polaris, quindi la trasformazione predetta li riporta nel phantom. L'errore è la norma della differenza.

Funzione: `svd_baseline_4pts`

Pre-condizioni: `landmark_pts_mm`: $[4, 3]$; `R_gt`, `t_gt_mm`; `rng`; `noise_sigma_mm`.

Post-condizioni: Coppia (R, t) stimata dalla baseline.

Spiegazione:

Simula l'acquisizione di 4 landmark noti con rumore gaussiano e stima la trasformazione tramite SVD. Restituisce la trasformazione stimata.

Funzione: `test_evaluation_math`

Pre-condizioni:

Post-condizioni:

Spiegazione:

Test unitario che verifica la correttezza delle funzioni di valutazione: SVD senza rumore, TRE nullo, errore di 1 mm e rotazione di 90 gradi.

Funzione: `evaluate`

Pre-condizioni: `args`: namespace degli argomenti.

Post-condizioni: Dizionario di metriche riassuntive.

Spiegazione:

Carica il checkpoint, ricostruisce il modello, crea il dataset di valutazione ed esegue la valutazione. Calcola metriche per DCP e baseline (se i landmark sono definiti). Salva i risultati in `eval.log` e restituisce un dizionario con statistiche (media, std, mediana, p95, max) per ciascuna metrica.

Funzione: `main`

Pre-condizioni:

Post-condizioni:

Spiegazione:

Punto di ingresso per la valutazione. Gestisce il parsing degli argomenti e chiama `evaluate`.

10 File test_correspondence.py

Funzione: small_args

Pre-condizioni: `**overrides`.

Post-condizioni: Namespace con parametri di default per test.

Spiegazione:

Crea un namespace con parametri ridotti per eseguire test veloci.

Classe: CorrespondenceTests

Descrizione: Suite di test unitari per la loss di corrispondenza, il modello DCP e il dataset.

Metodi:

- `setUpClass` — *Pre:* ; *Post:* Imposta il numero di thread di PyTorch a 1.
- `setUp` — *Pre:* ; *Post:* Inizializza tensori di test e seed.
- `loss` — *Pre:* logits, src, tgt, scale.; *Post:* Wrapper per `geometric_correspondence_loss`.
- `test_forward_and_checkpoint` — *Pre:* ; *Post:* Verifica il forward del modello e il salvataggio/caricamento dello stato.
- `test_geometric_optimum_and_scale` — *Pre:* ; *Post:* Verifica che la loss sia minima in corrispondenza della soluzione geometrica e che sia invariante alla scala.
- `test_gradients_only_through_prediction` — *Pre:* ; *Post:* Verifica che i gradienti fluiscano solo attraverso i logits.
- `test_permutation_invariance` — *Pre:* ; *Post:* Verifica l'invarianza all'ordine dei punti.
- `test_backward_with_and_without_cycle` — *Pre:* ; *Post:* Verifica la backpropagation con e senza cycle consistency.
- `test_epoch_metrics_and_disabled_loss` — *Pre:* ; *Post:* Verifica le metriche di training/validation e il comportamento con `corr_weight=0`.
- `test_dataset_reference_and_validation` — *Pre:* ; *Post:* Verifica la generazione del dataset e la validazione dei parametri.
- `test_evaluation_math` — *Pre:* ; *Post:* Esegue il test di `evaluate.test_evaluation_math`.

11 File unify.py

Script che carica tutti i file STL presenti in una directory, li concatena in un'unica mesh e la esporta come `phantom_combined.stl`. Stampa i bounds e gli extents della mesh risultante.

Listing 1: unify.py

```
1 import trimesh
2 import os
3
4 stl_dir = "ModelloPhantom"
5
6 meshes = []
7 for f in os.listdir(stl_dir):
8     if f.endswith('.stl'):
9         m = trimesh.load(os.path.join(stl_dir, f), force='mesh')
10        meshes.append(m)
11
12 combined = trimesh.util.concatenate(meshes)
13 combined.export("phantom_combined.stl")
14 print(f"Bounds: {combined.bounds}")
15 print(f"Extents: {combined.extents}")
```

Riferimenti bibliografici

- [1] Y. Wang, J. M. Solomon, “Deep Closest Point: Learning Representations for Point Cloud Registration,” ICCV 2019. arXiv:1905.03304. Codice: <https://github.com/WangYueFt/dcp>.
- [2] Y. Aoki, H. Goforth, R. A. Srivatsan, S. Lucey, “PointNetLK: Robust & Efficient Point Cloud Registration Using PointNet,” CVPR 2019. arXiv:1903.05711.
- [3] C. Choy, J. Park, V. Koltun, “Fully Convolutional Geometric Features,” ICCV 2019.
- [4] A. Vaswani et al., “Attention is All You Need,” NeurIPS 2017.
- [5] K. S. Arun, T. S. Huang, S. D. Blostein, “Least-Squares Fitting of Two 3-D Point Sets,” IEEE TPAMI, 9(5), 1987.